Week 5 APP DEV-2

2025 Mar16 Quiz-2

1. [Source Q10; MCQ]

Consider the following javascript code.

```
async function fetchData() {
    try {
        const response = await fetch('https://api.example.com/data');
        if (!response.ok) throw new Error('HTTP error');
        const data = await response.json();
        return data;
    } catch (error) {
        if (error.name === 'TypeError') {
            return { status: 'network error' };
        }
        return { status: 'http error' };
    }
}
```

When the function is called what will be returned if the server is **unreachable**?

(a) { status: 'http error' }

(b) { status: 'network error' }

(c) undefined

(d) The promise will remain pending

[Source ans: b]

2. [Source Q15; MCQ]

Consider the below Vue router setup.

Note: Assume that the API "/api/users/userId" is operational and returns the user's data including name.

```
const UserProfile = {
  template: `<div><h2>User Profile</h2><p>{{ user.name }}</p></div>`,
  data() {
    return {
      user: {}
    };
  },
  created() {
    this.fetchUserData();
  },
  methods: {
    async fetchUserData() {
      const userId = this.$route.query.userId;
      const response = await fetch(`/api/users/${userId}`);
      this.user = await response.json();
    }
  }
};

const routes = [
  { path: '/user', component: UserProfile }
];

const router = new VueRouter({
  routes
});
```

What will happen if the user navigates to "/user?userId=123"?

(a) The component will display the user's name (fetched from API) correctly asexpected.

(b) The component will show because the API call will never be triggered.

(c) The component will always show an empty profile because the userId queryparameter is not reactive.

(d) The component will throw an error because this.$route.query is not defined.

[Source ans: a]

2025 Apr13 ET

3. [Source Q16; MCQ]

Consider the following JavaScript code.

```
async function fetchData() {
    try {
        const response = await fetch('https://api.example.com/data');
        if (!response.ok) throw new Error('HTTP error');
        const data = response.json();
        return data;
    } catch (error) {
        if (error.name === 'TypeError') {
            return { status: 'network error' };
        }
        return { status: 'http error' };
    }
}
```

When the function is called, what will be returned if the request gives a 200 OK?

(a) { status: 'http error' }

(b) { status: 'network error' }

(c) JSON data

(d) The promise will remain pending

[Source ans: d]

4. [Source Q19; MCQ]

Consider the below CORS enabled flask app.

```
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "http://localhost:5000"}})

@app.route("/api/data")
def data():
      return {"message": "Hello from Flask"}
```

If a Vue.js frontend (simulation given below) at "http://localhost:3000" tries to fetch data, what will happen?

```
fetch("http://localhost:5000/api/data")
  .then(response => response.json())
  .then(data => console.log(data));
```

(a) The request will be blocked due to CORS policy.

(b) The request will succeed and return { "message": "Hello from Flask" }.

(c) The request will fail because Vue.js does not support CORS.

(d) The server will crash due to incorrect CORS settings.

[Source ans: a]

2025 Aug03 Quiz-2

5. [Source Q2; MCQ]

Suppose a client wants to make a **POST** request using the Fetch API to a public endpoint at `/api/submit`, which includes sending a **data** object as JSON in the request body.

```
const data = {
  name: "Aditya",
  score: 95,
};

fetch("/api/submit", {
  method: "POST",
  headers: {
    // CODE
  },
  body: JSON.stringify(data),
});
```

Which headers are **necessary** for this request to succeed, (inside CODE)?

(a)
```
'Content-Type': 'application/json'
```

(b)
```
'Content-Type': 'multipart/form-data'
```

(c)
```
'Content-Type': 'application/text'
```

(d) `'Accept': '*/*'`

[Source ans: a]

6. [Source Q6; MCQ]

Consider the following code snippet.

```
const response = {
  meta: () =>
    Promise.resolve([
      { id: 1, score: 20 },
      { id: 2, score: 40 },
    ]),
  json: () =>
    Promise.resolve([
      { name: "Alpha", score: 40 },
      { name: "Beta", score: 30 },
      { name: "Charlie", score: 50 },
      { name: "Delta" }, // no score
    ]),
};

fetchData();

function fetchData() {
  Promise.resolve(response)
    .then((res) => res.json())
    .then((data) => {
      const total = data
        .map((p) => parseInt(p.score) || 0)
        .filter((score) => score >= 40)
        .reduce((acc, score) => acc + score, 0);
      console.log("Total Score:", total);
    });
}
```

What will be logged to the console?

(a) Total Score: 80

(b) Total Score: 90

(c) Total Score: 120

(d) Total Score: NaN

[Source ans: b]

Shared context (source Q16)

You are given the following HTML and JavaScript code that uses Vue 2 via CDN along with Vue Router. Read the code carefully and answer the given subquestions.

Index.html

```
<div id="app">
    <router-link to="/home">Home</router-link>
    <router-link to="/about">About</router-link>
    <router-view></router-view>
</div>
```

app.js

```
const Home = {
      template: `
        <div>
          <h2>Home Page</h2>
          <p v-if="loading">Loading data...</p>
          <p v-show="!loading">{{ message }}</p>
          <button @click="fetchData">Fetch</button>
        </div>
      `,
      data() {
        return {
          message: '',
          loading: false
        };
      },
      methods: {
        async fetchData() {
          this.loading = true;
          await this.simulateApiCall().then((res) => {
            this.message = res;
            this.loading = false;
          });
        },
        simulateApiCall() {
          return new Promise((resolve) => {
            setTimeout(() => resolve("Fetched Data from API!"), 1000);
          });
        }
      }
    };
```

```
    const About = {
      template: '<div><h2>About Page</h2></div>'
    };
    const router = new VueRouter({
      routes: [
        { path: '/home', component: Home },
        { path: '/about', component: About }
      ]
});

new Vue({
   el: '#app',
   router
});
```

7. [Source Q17; MCQ]

What will be the output when the "Fetch" button is clicked on the `/home` route?

(a) The page immediately shows "**Fetched Data from API!**" with no delay.

(b) The message "**Loading data...**" is shown for ~1 second, then replaced with "**Fetched Data from API!**".

(c) Both "**Loading data...**" and "**Fetched Data from API!**" appear together.

(d) Nothing happens because `v-show` and `v-if` cannot work together.

[Source ans: b]

8. [Source Q18; MCQ]

Suppose the button is clicked multiple times quickly, the message area flickers unexpectedly. How can you improve the code to ensure proper async behavior and prevent overlapping fetches?

(a) Add `v-else` after `v-if` instead of v-show.

(b)

Replace

`await this.simulateApiCall().then(...);` with

```
const res = await this.simulateApiCall(); this.message = res;
```

(c)

Add a check to disable the button while loading;

```
<button @click="fetchData" :disabled="loading">Fetch</button>
```

(d)

Use v-html instead of v-show for message rendering.

[Source ans: c]

2025 Aug31 ET

9. [Source Q5; MCQ]

What is the primary purpose of CORS (Cross-Origin Resource Sharing)?

(a) To encrypt data between client and server

(b) To compress HTTP responses

(c) To authenticate users across applications

(d) To control which domains can access resources

[Source ans: d]

10. [Source Q28; MCQ]

Given the Flask app:

```
from flask import Flask
from flask_cors import CORS

app = Flask(__name__)
CORS(app, resources={r"/api/*": {"origins": "http://localhost:3000"}})

@app.route('/api/data')
def data():
    return {"msg": "Success"}
```

frontend code:

```
fetch("http://localhost:5000/api/data")
  .then(res => res.json())
  .then(data => console.log(data));
```

What will happen when run from a Vue app hosted at http://localhost:5173?

(a) Request succeeds

(b) CORS error occurs

(c) 404 error

(d) The server crashes

[Source ans: b]

2025 Dec18 ET

11. [Source Q12; MCQ]

What happens to the fetch call when it gets a 404 status code (page not found)?

(a) Promise rejects automatically

(b) Promise resolves and `response.ok = true`

(c) Promise resolves and `response.ok = false`

(d) Promise stays as pending

[Source ans: c]

2026 Apr06 Quiz-2

12. [Source Q5; MCQ]

Match the following web dev concept given below:

Concept / Issue	Description
A. "Chatty" nature of REST APIs	1. Static HTML is enhanced with event handlers after initial page load
B. Limitation of REST as a query mechanism	2. May require multiple API calls to fetch data for a single view
C. JavaScript Hydration	3. Focuses on decoupling storage, logic, and presentation using APIs
D. JAMStack approach	4. Does not allow clients to specify exactly what data is required in one request

Which of the following options correctly matches Concept / Issue and Description?

(a) A→2, B→4, C→1, D→3

(b) A→1, B→2, C→3, D→4

(c) A→2, B→1, C→4, D→3

(d) A→4, B→2, C→1, D→3

[Source ans: a]

2026 May06 ET

13. [Source Q27; MSQ]

Prerna and Swathy are using a web application built with Vue.js for the BS Degree program. When the page loads, the sidebar content appears instantly• However, when they click on an Assignments link, the main content takes noticeable time to load • A simplified version of the component is shown below:

```
export default {
  data() {
    return {
      assignments: []
    }
  },
  created() {
    fetch("/api/assignments")
      .then(res => res.json())
      .then(data => {
        this.assignments = data;
      });
  }
}
```

Which of the following statements correctly explain why the Assignments page takes time to load?

(a) The sidebar is likely part of a parent component and is already rendered before the API call

(b) The Assignments component waits for the API response before displaying data

(c) Vue blocks rendering of all components until API calls complete

(d) The delay is caused because data is fetched asynchronously after the component is created

[Source ans: a, b, d]

2026 Aug14 Quiz-2

14. [Source Q3; MCQ]

Point out the error with the following code that is used to display user data?

```
async function getUser() {
 const response = fetch("https://api.example.com/user/1");

 if (!response.ok) {
   throw new Error("Request failed");
 }

 const data = await response.json();
 console.log(data.name);
}

getUser();
```

(a) `fetch()` should use POST method by default, not GET

(b) `response.ok` and `response.json()` will throw errors because response is a `Promise`

(c) `.json()` cannot be called on a `fetch` response; use `.text()` instead

(d) `fetch()` does not work inside `async` functions

[Source ans: b]